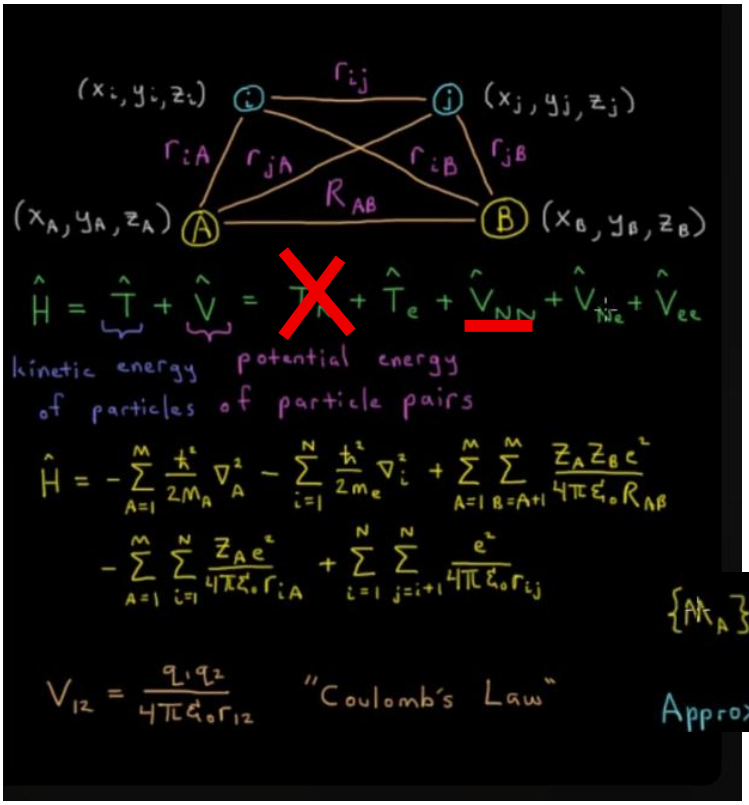
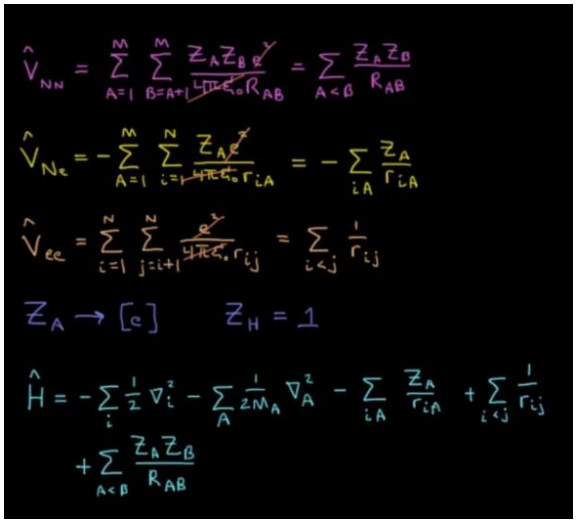
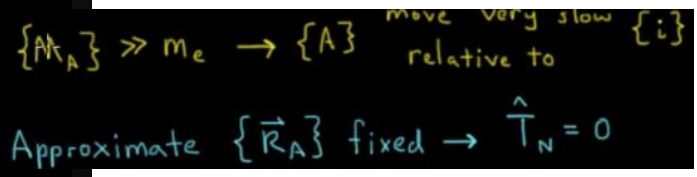
Notes on Hartree Fock theory

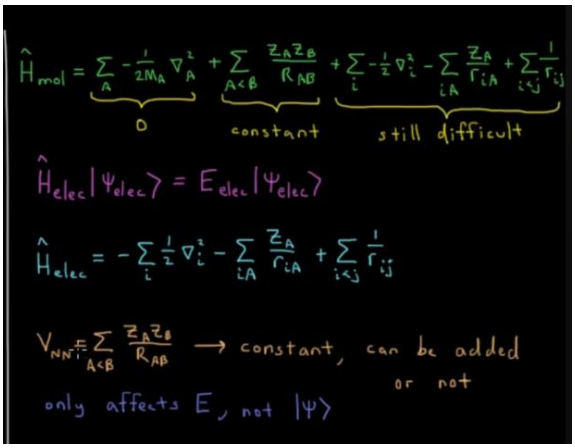


Atomic units simplify it to

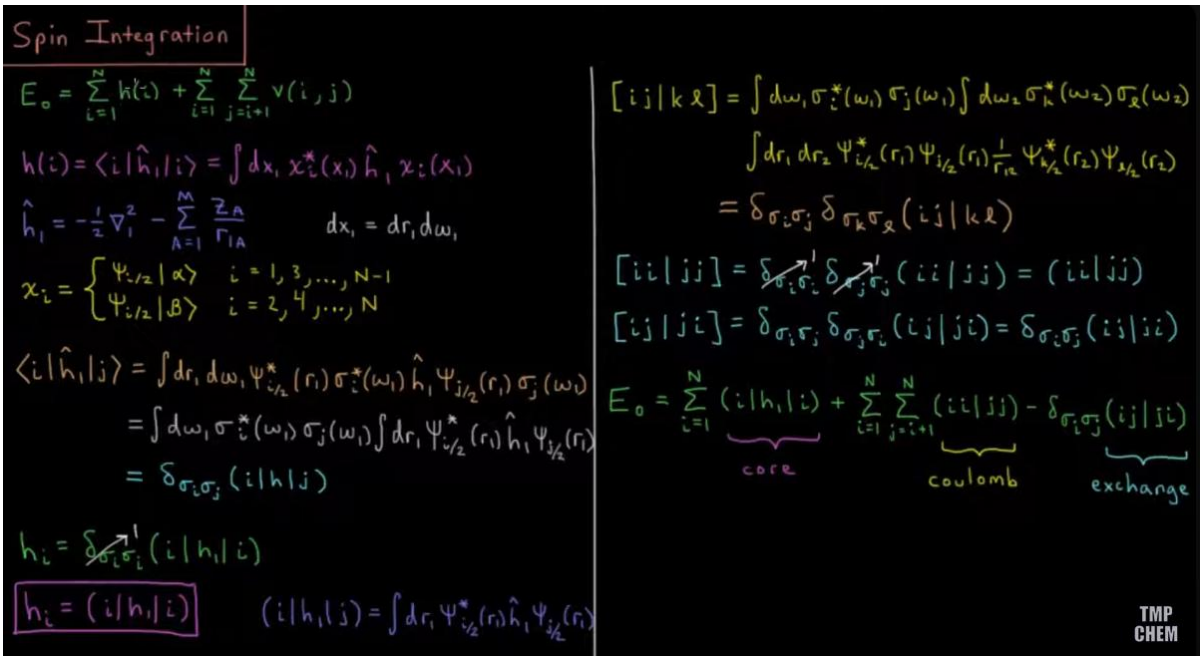




Nuclei fixed → no Tn, Vnn constant



Vnn only affects total energy but not shape of the wave functions (so can be omitted and added in final step)



E0 (of ground state determinant) = SUM,i h(i) + SUM,ij V(i,j) interaction energy

- h(i): core energy = kinetic energy of electron + attraction to all nuclei
- V(i,j): interaction energy

Design plan

Main program hartree_fock

- calls functions
- along the lines of:

```
Ask for and read input
Declare variables

Call initialize_system

Call compute_integrals

Call build_density

D = 0
E_old = 0
n = 0

do iter = 1, max_iter

  call build_fock(...)
  call solve_eigen(...)
  call build_density(...)
  E_new = compute_energy(...)

  if (...) < threshold    exit                    (probably call check_convergence)

  else

    E_old = E_new
    n = n+1
end do

EHF = E_new + nuclear repulsion

Output results
```

Subroutines / functions

- initialize_system
  - subroutine initialize_system(...)
  - atoms, electrons Ne, Nocc, basis set, NOA
- compute_integrals *(2 elec. Integrals (g) with generate_2int)*
  - subroutine compute_integrals(S, H_core, g)
  - compute S H_core, g via library
- build_density
  - subroutine build_density(C, D, Nocc)
  - D = sum over Nocc
- build_fock
  - subroutine build_fock(F, H_core, D, g)
- solve_eigenproblem *(with solve_genev)*
  - subroutine solve_eigen(F, S, C, eps)
- compute_energy
  - function compute_energy(D, H_core, F) result(E)
- check_convergence
  - ...

Data types

To be defined

```
integer :: Ne, iter, max_iter
real(8) :: E_old, E_new, threshold
real(8), allocatable :: H_core(:,:)
```

Already existing

```
integer  :: n_AO, n_occ

integer  :: kappa, lambda

real(8)  :: E_HF

real(8), allocatable :: F(:,:),V(:,:),T(:,:),S(:,:), C(:,:), eps(:), D(:,:)

real(8), allocatable :: ao_integrals (:,:,:,:)
```

Flow chart

>>>>>> next page

START

Read input
- # and type of atoms
- xyz coordinates

Determine system size
- # electrons (Ne = ΣZ)
- # occupied orbitals (Nocc = Ne/2)
- Define set of basis functions χ(r). (per atom)
- # basis functions in total (NAO)

Compute integrals
- Overlap matrix (S)
- 2 elec. Integrals (g) (with *generate_2int*)
- Core Hamiltonian Matrix (h_core)

PREPARATION

Initial guess
- Density matrix: D = 0
- Solve: h_core C = S C_e (initial F = h_core)
- Select Nocc lowest eigenvectors (C_e)

Initialize loop values
- Energy: E_old = 0
- Iteration: n = 0

SRC LOOP: solve eigenproblem and find D
- Build Fock matrix F (using h_core, D, G) eq. 3
- Solve generalised eigenvalue problem:
  F C = S C_e (with *solve_genev*)
- Compute new D (from Nocc lowest C_e) eq.2
- Compute energy E_new eq. 4

Convergenge Check
- abs(E_old – E_new) < threshold ?
  - YES → exit
  - NO → update loop variables:
    E_old = E_new
    n = n+1

n = n+1

Compute final HF energy
- incl. nuclear repulsion (Vnn) eq. 4

MATRIX CALCULATIONS

Output results
- Final EHF
- # iterations

RESULTS

END

NOTES

Start with Be-He dimer as basis set and extend (H: 5s, 3p, 1d, non-H: 3s, see exponents in exercise)

e.g. NAO = 7 for H2O (3+3+1)

h_core: electron kinetic energy (Te) + nuclear attraction (Ven) (no inter electronic interactions (Vee))

(step 1 & 2 from exercise)

Solve = diagonalize

(step 3,4,5 from exercise)

SRC = Self-Consistent Field

LOOP:

D → F → C → D until converge

More robust: taking the norm of the commutator (i.e. [F, D] = FD − DF) of the Fock and density matrices

Should have max no. of iterations (do iter = 1, max_iter)